

CZEMU TAK JEST

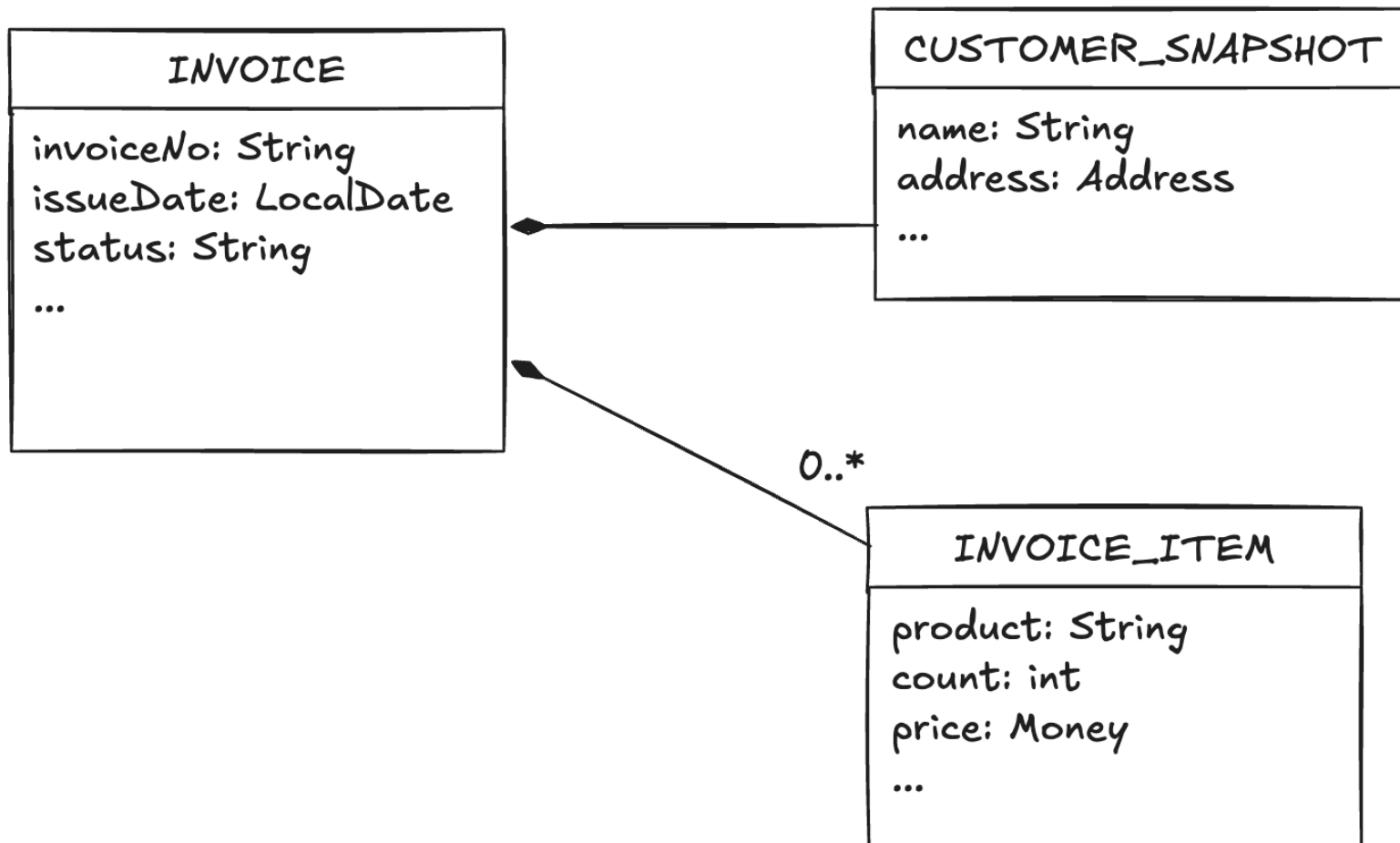


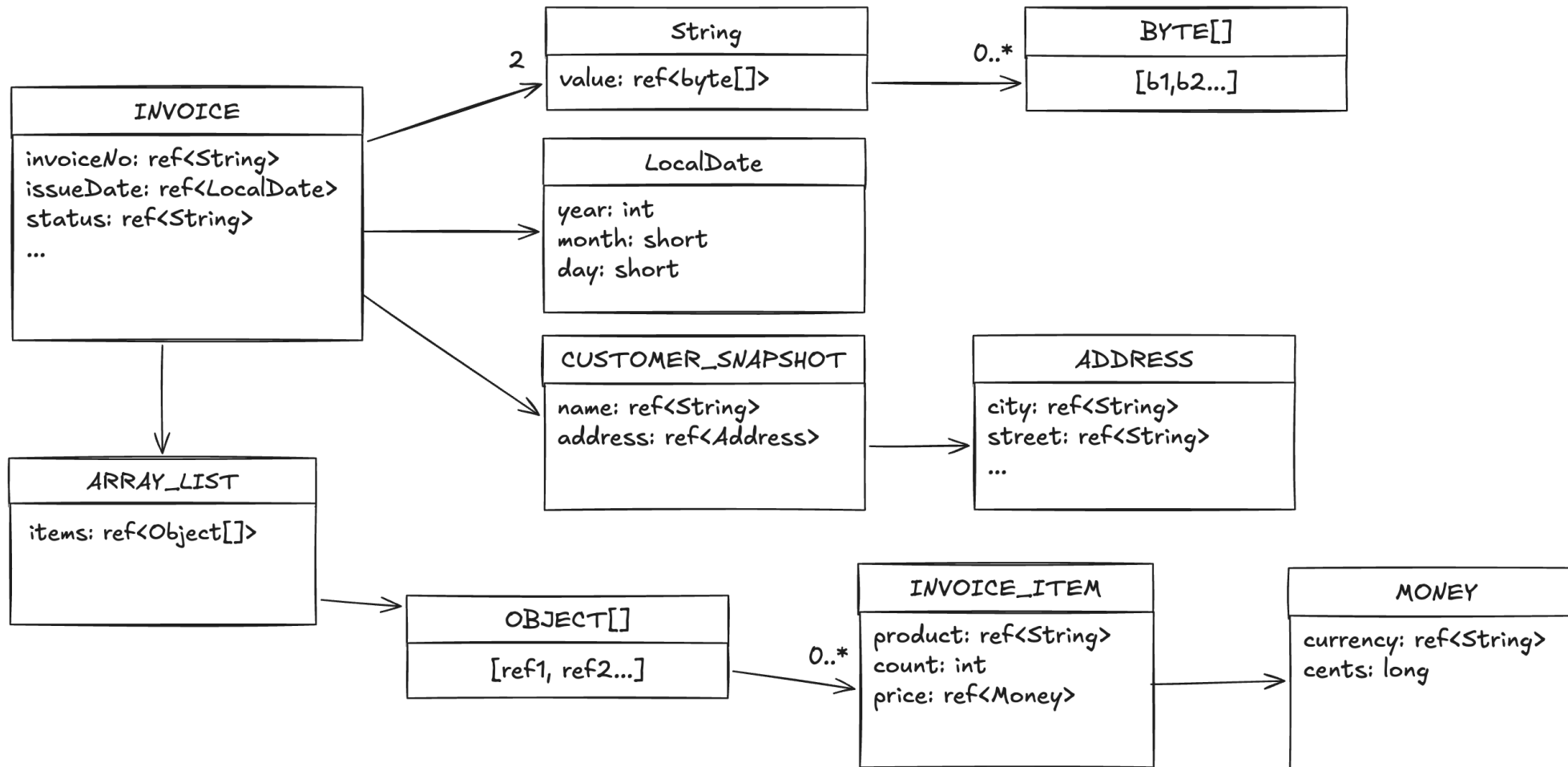
Obiekty bez tożsamości Value Class w JEP 401

Leszek Leszczyński

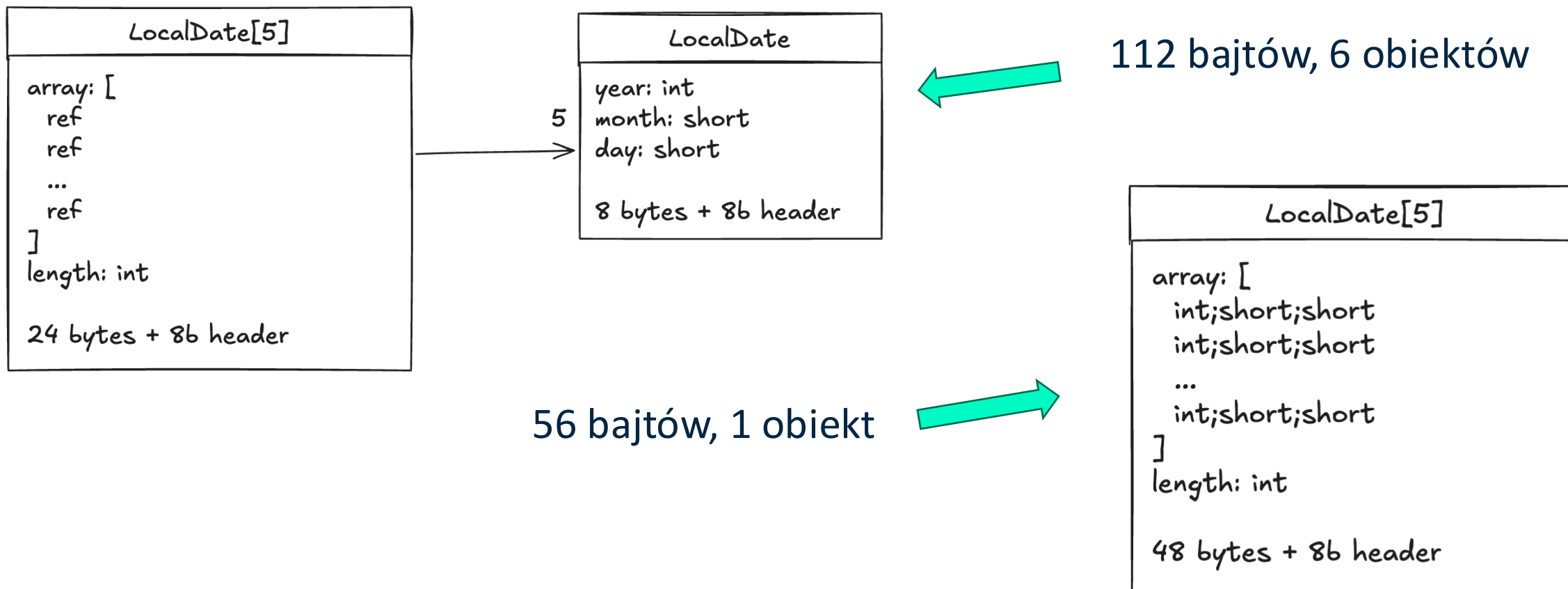


<https://github.com/LeszekLeszczynski/confitura2026>

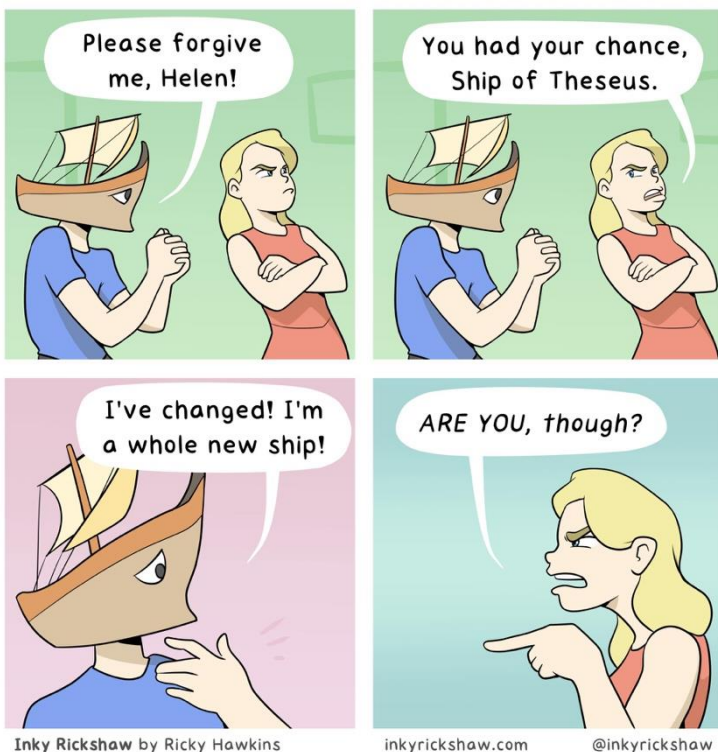




„Object identity is expensive at run time”



„Immutable data does not need identity”



Mutable data – obiekt zachowuje „tożsamość” mimo zmiany danych

Immutable data – dane niezmiennie przez cały czas istnienia obiektu. Zmiana danych – nowy obiekt. Dwa niemutowalne obiekty o takich samych danych powinny być w praktyce nierozróżnialne i wymienne, a więc nie potrzebują „tożsamości”.

JEP 401 – INTRODUCING VALUE OBJECTS

„ We introduce *value objects* to model simple* immutable data. A value object is an instance of a *value class*, declared with the value modifier. Classes without the value modifier are *identity classes*, and their instances are *identity objects*.“

„Traditionally, references are represented in a JVM as pointers to memory locations, thus encoding the unique identity of each object.“

„We can save memory and improve performance by using value objects for immutable data. Because a program cannot distinguish two value objects with identical field values, not even with the == operator, a JVM is able to change how a value object is laid out in memory without affecting the program. A JVM could, e.g., store the fields of a value object on the stack or even in CPU registers, rather than the heap.“

* clear underestimation of people creativity



ANATOMIA VALUE CLASS

Value class w praktyce – niemutowalne pola i brak identity

Kandydat na value class:

- immutable – niemutowalny
- interchangeable – wymienny

„By opting out of identity, developers opt in to a programming model that enables the best of both worlds: the abstraction of classes, with the simplicity and performance benefits of primitives.”

java

```
value record Point(int x, int y) {}  
// |                               |  
// |                               |   pola: niejawnie final  
// |                               |  
// |   modyfikator: klasa niejawnie final, brak tożsamości, == po stanie
```

Następujące klasy automatycznie stają się value classami w SDK 28 Preview (wybrane):

Package	Classes
java.lang	Integer, Long, Float, Double, Byte, Short, Character, Boolean
java.util	Optional, OptionalInt, OptionalLong, OptionalDouble
java.time	LocalDate, LocalTime, LocalDateTime, ZonedDateTime, Duration

Czemu nie String?

Cache the hash code for the string

```
@Stable
```

```
private int hash; // Default to 0
```

Cache if the hash has been calculated as actually being zero, enabling us to avoid recalculating this. This field is `_not_` annotated `@Stable` as the `hashCode()` method reads the field `hash` first anyhow and if `hash` is the default zero value, is not trusted.

```
private boolean hashIsZero; // Default to false;
```

Value Class nie ma tożsamości – synchronizacja nie działa

```
jshell> Integer riskId = 1500100900;
riskId ==> 1500100900

jshell> synchronized (riskId) { System.out.println("synchronized block"); }
| Error:
| unexpected type
|   required: identity class
|   found:    java.lang.Integer
| synchronized (riskId) { System.out.println("synchronized block"); }
| ^-----^
```

Value Class nie ma tożsamości – nie może być kluczem w WeakHashMap

```
jshell> var cache = new WeakHashMap<Object, String>();
cache ==> {}

jshell> cache.put(riskId, "kopytko")
| Warning:
| use of a value-based class with an operation that expects reliable identity
| cache.put(riskId, "kopytko")
|   ^-----^
| Exception java.lang.IdentityException: java.lang.Integer is not an identity class
|   at Objects.requireIdentity (Objects.java:243)
|   at WeakHashMap.put (WeakHashMap.java:480)
|   at (#46:1)
```

Value Class nie ma tożsamości – nie można utworzyć WeakReference

```
jshell> var wr = new WeakReference<>(riskId);
| Warning:
| use of a value-based class with an operation that expects reliable identity
| var wr = new WeakReference<>(riskId);
|   ^-----^
| Warning:
| use of a value-based class with an operation that expects reliable identity
| var wr = new WeakReference<>(riskId);
|   ^-----^
| Warning:
| use of a value-based class with an operation that expects reliable identity
| var wr = new WeakReference<>(riskId);
|   ^-----^
| Exception java.lang.IdentityException: java.lang.Integer is not an identity class
|   at Objects.requireIdentity (Objects.java:243)
|   at Reference.<init> (Reference.java:543)
|   at Reference.<init> (Reference.java:538)
|   at WeakReference.<init> (WeakReference.java:69)
|   at do_it$Aux (#44:1)
|   at (#44:1)
```

Pola w Value Class są bardziej final niż final w Identity Class

```
class IdP { final int x; IdP(int x) { this.x = x; } }  
value class ValP { int x; ValP(int x) { this.x = x; } }  
  
var fi = IdP.class.getDeclaredField("x");  
fi.setAccessible(true);  
var ip = new IdP(1);  
fi.setInt(ip, 42);  
assert ip.x == 42;  
  
var fv = ValP.class.getDeclaredField("x");  
fv.setAccessible(true);  
var vp = new ValP(1);  
fv.setInt(vp, 42);
```

Pola final w Identity Class można zmieniać refleksją.

Pól final w Value Class nie można zmieniać refleksją.

```
Exception java.lang.IllegalAccessException: Can not set final int field REPL.$JShell$15$ValP.x to (int)42  
    at FieldAccessorImpl.throwFinalFieldIllegalAccessException (FieldAccessorImpl.java:132)  
    at FieldAccessorImpl.throwFinalFieldIllegalAccessException (FieldAccessorImpl.java:156)  
    at MethodHandleIntegerFieldAccessorImpl.setInt (MethodHandleIntegerFieldAccessorImpl.java:160)  
    at Field.lambda$setInt$0 (Field.java:1211)  
    at Field.setFinal (Field.java:1532)  
    at Field.setInt (Field.java:1211)  
    at (#78:1)
```

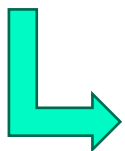
Oczekuj niespodzianek nie tylko we własnym kodzie!

```
import org.apache.commons.lang3.builder.ReflectionToStringBuilder;

static class Plain {
    int count = 3;
    long total = 1000;
    String name = "demo";
}

ReflectionToStringBuilder.toString(plain, ToStringStyle.SHORT_PREFIX_STYLE)
```

(Apache commons-lang3 3.20.0 – 12.11.2025 – najnowsza)



IdentityException:

```
java.lang.Integer is not an identity class
@ ToStringStyle.register:608
```

```
value record Money(long cents) {}
value record InvoiceItem(String product, int count, Money price) {}

List<InvoiceItem> items = List.of(
    new InvoiceItem("Śruba M8", 100, new Money(25)),
    new InvoiceItem("Nakrętka M8", 100, new Money(10)),
    new InvoiceItem("Śruba M8", 100, new Money(25)) // osobna pozycja, druga dostawa
);

// "nie licz dwa razy tego samego obiektu" (np. gdy pozycja jest podpięta w kilku miejscach grafu)
Set<Object> visited = Collections.newSetFromMap(new IdentityHashMap<>());

long total = 0;
for (var it : items) {
    if (!visited.add(it)) continue;
    total += it.count() * it.price().cents();
}
total
```

UWAGA NA IdentityHashMap

Oczekuj niespodzianek nie tylko we własnym kodzie!

```
import org.ehcache.sizeof.SizeOf;

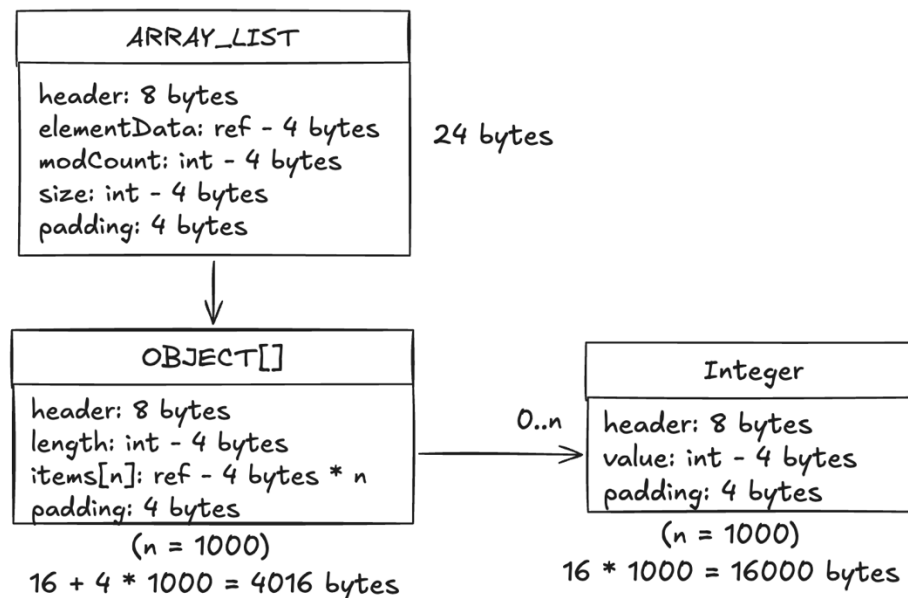
static List<Integer> boxedList(int n) {
    var list = new ArrayList<Integer>(n);
    IntStream.range(0, n).forEach(i -> list.add(Integer.valueOf(1000)));
    return list;
}

var list = boxedList(1000);
var size = SizeOf.newInstance().deepSizeOf(list);
```

(ehcache 3.12.0 – 3.4.2026 - najnowsza)

Java 28

```
=== deepSizeOf(list of 1000 separately boxed Integer.valueOf(1000)) ===
reported by sizeof:          20 040 bytes
```



Java 28 --enable-preview

```
=== deepSizeOf(list of 1000 separately boxed Integer.valueOf(1000)) ===
reported by sizeof:          4 056 bytes
```

Integer(1000) -> policzone tylko raz!



```
value class Iban {  
    private String value;           // implicite final  
  
    // Iban(String raw) {  
    //     value = normalize(raw);  
    //     isValid();               // błąd kompilacji!  
    //}  
  
    Iban(String raw) {  
        value = normalize(raw);    // OK: metoda statyczna  
        super();                  // koniec early construction  
        isValid();                 // OK  
    }  
  
    private static String normalize(String s) { return s.replace(" ", "").toUpperCase(); }  
    boolean isValid() {  
        // TODO: validation, throw IllegalArgumentException if invalid  
        return true;  
    }  
}
```

Przed JEP 401

== porównuje **referencje**

```
jshell> LocalDate l1 = LocalDate.of(2026,9,26)
l1 ==> 2026-09-26

jshell> LocalDate l2 = LocalDate.of(2026,9,26)
l2 ==> 2026-09-26

jshell> l1 == l2
$11 ==> false
```

Porównanie przez == jest niezawodne tylko
gdy wszystkie pola to prymitywy lub value
objects



== W PRAKTYCE

Po JEP 401

== dla value class porównuje **wartości pól klasy**
== dla identity class porównuje **referencje**

```
jshell> LocalDate l1 = LocalDate.of(2026,9,26)
l1 ==> 2026-09-26

jshell> LocalDate l2 = LocalDate.of(2026,9,26)
l2 ==> 2026-09-26

jshell> l1 == l2
$3 ==> true
```

UWAGA!

```
jshell> var a = new Iban("PL61 1090 1014 0000 0712 1981 2874");
a ==> Iban@5ebc05f8

jshell> var b = new Iban("PL61 1090 1014 0000 0712 1981 2874");
b ==> Iban@1c5c9e60

jshell> a == b
$7 ==> false
```

```
abstract value class Measurement {           //dziedziczenie tylko po klasach abstrakcyjnych
    private double value;                    // pola implicite final
    protected Measurement(double value) { this.value = value; }
    public double value() { return value; }
    public abstract String unit();
}

enum Scale { C, F, K }

value class Temperature extends Measurement {
    private Scale scale;                    // implicite final

    Temperature(double value, Scale scale) {
        this.scale = scale;                // pola ustawiamy przed super()
        super(value);
    }
    public String unit() { return "°" + scale; }
}
```

2014: start Projektu Valhalla. Hasło: „*codes like a class, works like an int*”.

2017–2019: prototypy (Minimal Value Types, L-World). Nazwy się zmieniają: *value types* → *inline classes* → *primitive classes*.

2020: powstaje JEP 401.

2021: JEP 390 (JDK 16): ostrzeżenia dla klas *value-based*, konstruktory wrapperów przeznaczone do usunięcia.

2022–2023: model „trzech koszyków” upraszcza się do dwóch: identity i value. Nullowalność ląduje w osobnym JEP-ie.

2025: JDK 25 dostarcza klocki: Flexible Constructor Bodies (JEP 513) i Compact Object Headers (JEP 519).

czerwiec 2026: JEP 401 zintegrowany z mainline OpenJDK

marzec 2027: JDK 28, value classes jako **preview** (`--enable-preview`).

dalej: preview prawdopodobnie także w kolejnym LTS; potem typy null-restricted i specjalizacja generyków.



<https://github.com/openjdk/jdk/pull/31120>



I Am Developer
@iamdeveloper

Follow



10 lines of code = 10 issues.

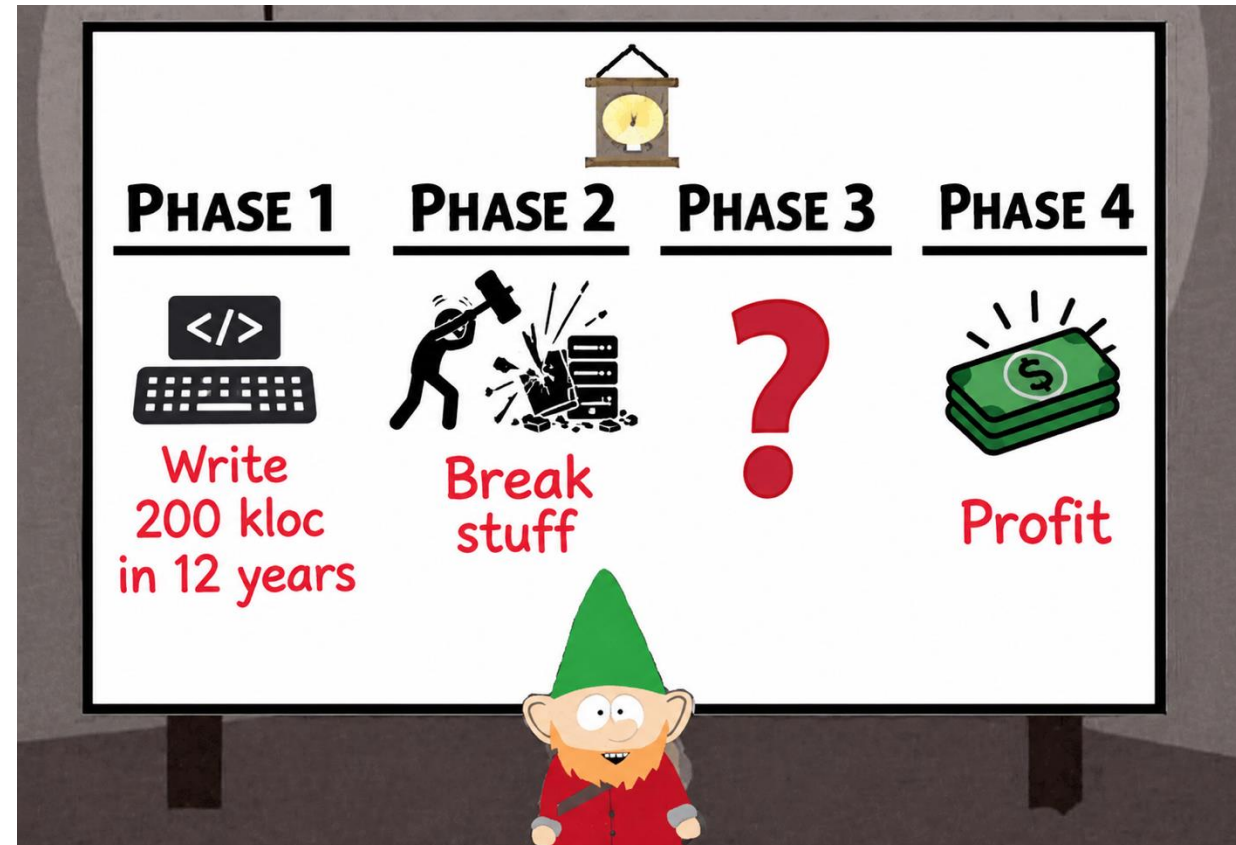
500 lines of code = "looks fine."

Code reviews.

1:58 AM - 5 Nov 2013

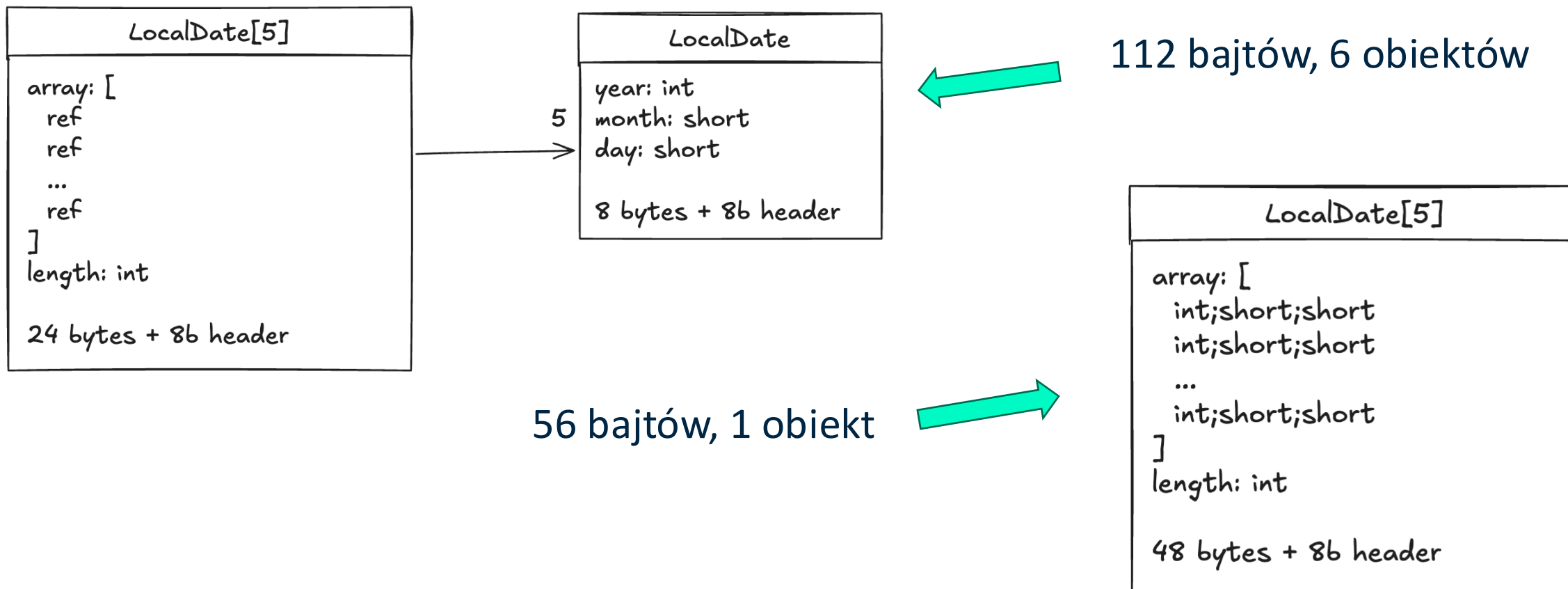
12 lat, 200 tysięcy linii kodu...

I wszystko to dla ==, które i tak nie zawsze działa?



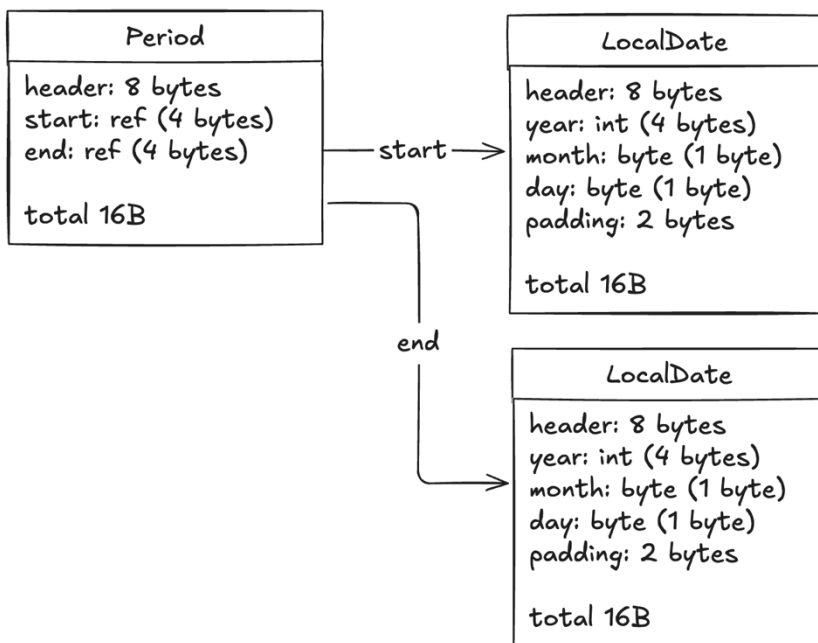
FLATTENING

JVM **może** (ale nie musi) zapisać obiekt value w postaci spłaszczonej, bez referencji i bez narzutu.



```
class Period {  
    LocalDate start;  
    LocalDate end;  
}
```

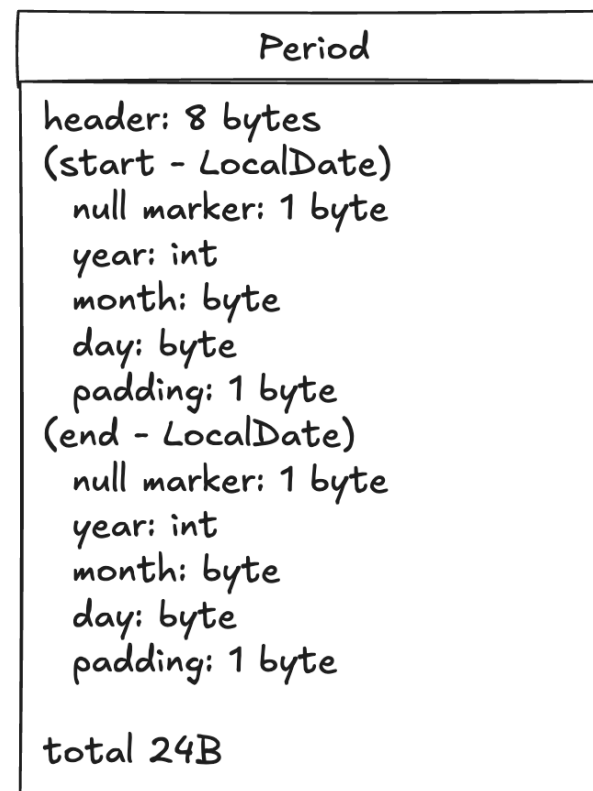
Bez JEP 401



48 bajtów, 3 obiekty

FLATTENING W POLACH

JEP 401

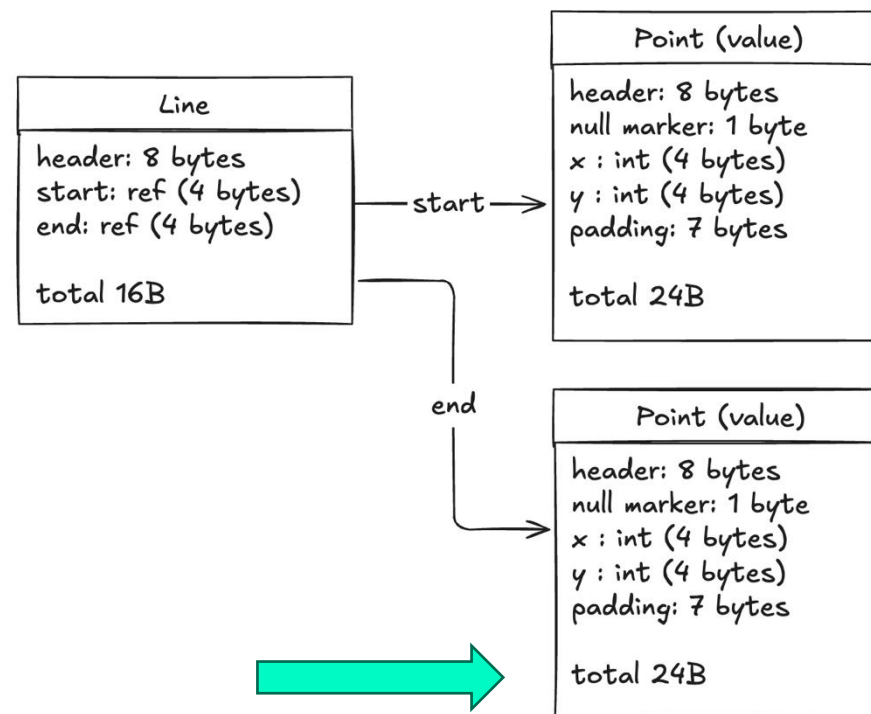
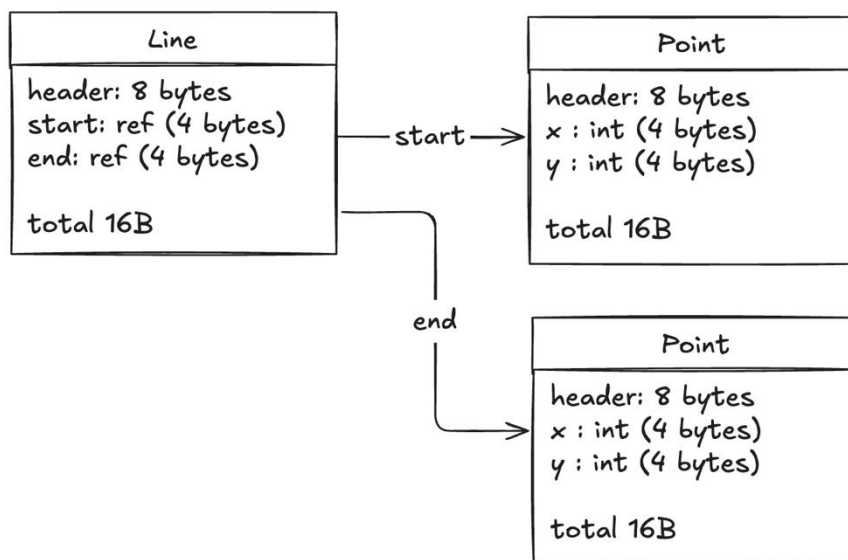


```
value class Point {  
    int x;  
    int y;  
}  
  
class Line {  
    Point start;  
    Point end;  
}
```

FLATTENING W POLACH - UWAGA

JEP 401

Bez JEP 401

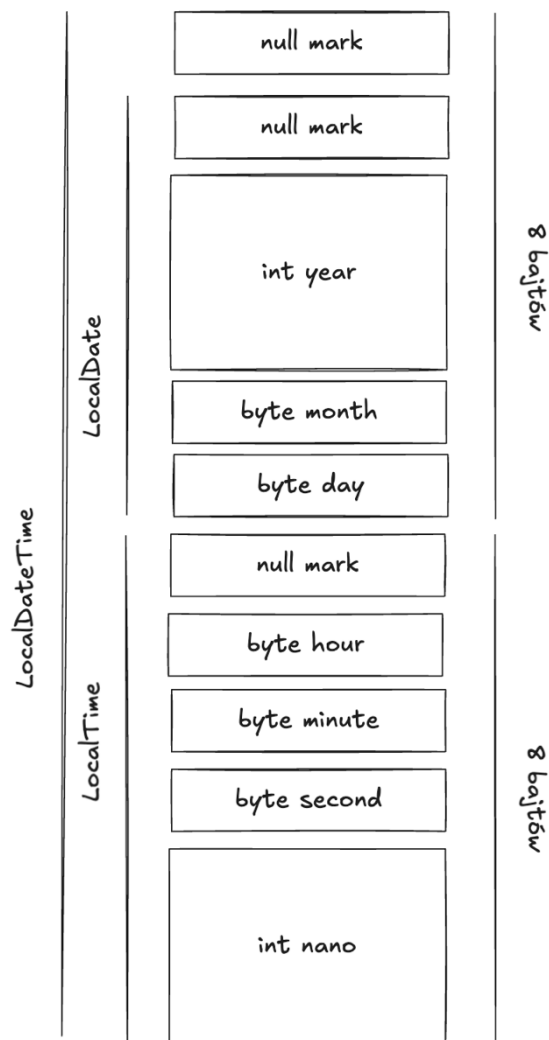


Obiekt jest za duży – zostaje zbuforowany na stercie. Limit rozmiaru to... 7 bajtów



SKĄD LIMIT 7 BAJTÓW?

Powyżej granicy atomowego zapisu – potencjalny **tearing**



```
@Data 2 usages
class Order {
    private LocalDateTime orderDateTime;

    public void updateOrderDT(LocalDateTime ldt) { no usages
        this.orderDateTime = ldt;
    }
}

// wątek 1 - order.updateOrderDT(LocalDateTime.of(dzisiaj rano))
// ⚡ wątek 2 - order.updateOrderDT(LocalDateTime.of(wczoraj wieczorem))
// --> orderDateTime == dzisiaj wieczorem
```

Jeżeli JVM może udowodnić, że pole nie zostanie **nigdy** nadpisane, może spłaszczać bez ograniczeń!

... co zapewnia JEP 539 – Strict Field initialization in the JVM

W Java 28 Preview – polami strict stają się

- Pola zadeklarowane w value classach
- Pola **rekordów**

<https://johan-sjolen.github.io/post/compiler-sympathy/compiler-sympathy/>



FLATTENING W PRAKTYCE

```
static class ClassicMoney {  
    long cents;  
    int currency;  
}  
  
static class ClassicTaxRate {  
    short basisPoints;  
}  
  
static class ClassicLineItem {  
    String sku;  
    Integer quantity;  
    ClassicMoney unitNet;  
    ClassicTaxRate taxRate;  
    LocalDate deliveredOn;  
}  
  
static class ClassicInvoice {  
    Long id;  
    String customer;  
    LocalDate issuedOn;  
    LocalDate dueOn;  
    List<ClassicLineItem> items;  
}
```

10 pozycji na fakturze – 44 obiekty, 1024 B

```
value record VMoney(  
    long cents,  
    int currency  
) {}  
  
value record VTaxRate(  
    short basisPoints  
) {}  
  
value record VLineItem(  
    String sku,  
    int quantity,  
    VMoney unitNet,  
    VTaxRate taxRate,  
    LocalDate deliveredOn  
) {}  
  
value record VInvoice(  
    long id,  
    String customer,  
    LocalDate issuedOn,  
    LocalDate dueOn,  
    VLineItem[] items  
) {}
```

10 pozycji na fakturze – 13 obiektów, 608 B

```
public static class ClassicAuthor {
    String name;
    String email;
    String bio;
    String avatarUrl;
}

public static class ClassicSection {
    String heading;
    String body;
    String anchor;
}

public static class ClassicArticle {
    String slug;
    String title;
    String summary;
    ClassicAuthor author;
    List<String> tags;
    List<ClassicSection> sections;
}
```

5 sekcji, 10 tagów – 324 B + stringi

FLATTENING FAIL

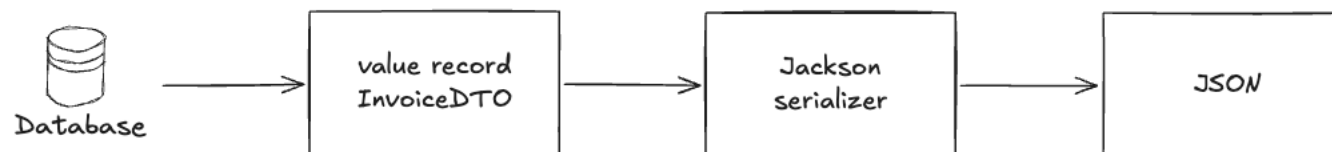
```
value record VAuthor(
    String name,
    String email,
    String bio,
    String avatarUrl
) {}

value record VSection(
    String heading,
    String body,
    String anchor
) {}

record VArticle(
    String slug,
    String title,
    String summary,
    VAuthor author,
    List<String> tags,
    List<VSection> sections
) {}
```

5 sekcji, 10 tagów – 320 B + stringi

ZANIM OTWORZYM SZAMPANA...



```
static final ObjectMapper MAPPER = JsonMapper.builder()
    .addModule(new JavaTimeModule())
    .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
    .visibility(PropertyAccessor.FIELD, Visibility.ANY) // th
    .build();
```

Run main | Debug main

```
public static void main(String[] args) throws Exception {
    System.out.println("Jackson " + MAPPER.version());
    var classic = InvoiceModels.classic(1);
    var value = ValueInvoiceModels.values(1);

    Demo.section("does it work?");
    var classicJson = MAPPER.writeValueAsString(classic);
    var valueJson = MAPPER.writeValueAsString(value);
}
```

```
=== bytes allocated per invoice, after warm-up (10000 rounds) ===
model          build model      serialize    deserialize
ClassicInvoice      1 055          3 672          9 029    (JSON: 1 582 bytes)
VInvoice            656          4 069         11 925    (JSON: 1 582 bytes)
```



- JVM **może** (ale nie musi) spłaszczać value objects w polach i tablicach
- Pola i elementy tablic mutowalne: **limit 7B danych** (+1 B null markera = 64 bity).
- Pola strict final (value classes, recordy): **bez limitu**.
- Przejście przez Object/T (generyki, kolekcje, refleksja, Jackson) oznacza **buforowanie na stercie**.
- Spłaszcza się sama wartość: pola String zostają referencjami.
- Spłaszczenie ≠ gwarancja mniejszego zużycia pamięci

```
value record Point(int x, int y) {  
    Point plus(Point o) { return new Point(x + o.x, y + o.y); }  
}  
  
static Point sum(int[] xs, int[] ys) {  
    Point s = new Point(0, 0);  
    for (int i = 0; i < xs.length; i++)  
        s = s.plus(new Point(xs[i], ys[i]));  
    return s;  
}
```



```
int s_x = 0, s_y = 0; // Point w dwóch rejestrach  
for (...) {  
    { s_x, s_y } = $plus(s_x, s_y, xs[i], ys[i]); // parametry i wynik rozbite na pola  
}  
return { s_x, s_y }; // zero obiektów na stercie
```

Pola value obiektu żyją w rejestrach i zmiennych lokalnych, a obiekt na stercie w ogóle nie powstaje!

Escape analysis potrafi to zrobić – częściowo - w granicach metody – z value obiektami działa przez granice metod.

Brak limitu rozmiaru; warunkiem spłaszczenia jest znajomość typu w trakcie kompilacji.

bytes allocated per plus(), after warm-up (10,000 calls × 100 rounds)

	inlining allowed	dontinline
identity record Point (8 B)	16.00 B	32.00 B
value record Point (8 B)	0.00 B	0.00 B
value record Big (32 B)	0.00 B	0.00 B
value Point via Object parameter	0.00 B	48.00 B

SKALARYZACJA - CD

```
value record MinMax(int min, int max) {}  
  
static MinMax minMax(int[] data) { ... return new MinMax(min, max); }
```

```
value record Money(long cents) {  
    Money plus(Money o) { return new Money(cents + o.cents); }  
    Money times(int percent) { return new Money(cents * percent / 100); }  
}  
  
Money total = Money.ZERO;  
for (var risk : risks) total = total.plus(risk.base().times(risk.loading()));
```

```
LocalDate.of(2026,9,26).plusDays(1).withDayOfMonth(1).plusMonths(3)
```

```
for (var m : items) total = total.plus(m);           // konkretny typ  
items.stream().reduce(Money.ZERO, Money::plus);     // BinaryOperator<T> → Object
```

Brak alokacji – para wartości zwrócona "za darmo"

Arytmetyka na obiektach domenowych
Fluent API bez alokacji

Uwaga na streamy!

UWAGA: skalaryzacja działa dopiero na poziomie C2 kompilacji!

```
=== bytes allocated per element, after warm-up (10000 elements x 100 rounds) ===  
                                     warmed up (C2)      C1 only  
1. return MinMax (value record)      0,00 B          24,00 B  
1. return MinMax (identity record)   0,00 B          16,00 B  
1. return two ints packed in a long  0,00 B           0,00 B  
2. pricing loop, value Money          0,00 B          72,00 B  
2. pricing loop, identity Money       16,00 B          32,00 B  
2. pricing loop, raw long              0,00 B          24,00 B  
3. for loop, total.plus(m)            0,00 B          24,00 B  
3. stream().reduce(ZERO, Money::plus) 24,01 B          24,01 B  
3. stream().map(...).reduce(...)      48,02 B          72,02 B  
3. stream().mapToLong(...).sum()       0,02 B          24,02 B
```

???

```
value record Money(long cents) {}

/** Money is a record component, so it sits inline: no Money object exists on the heap. */
record Risk(Money base, int loading) {}

static final List<Risk> RISKS =
    | IntStream.range(0, SIZE).mapToObj(i -> new Risk(new Money(1000 + i), 100)).toList();

static long sink;

/** One pass: read the flattened Money out of every Risk and add up the cents. */
static long readAll() {
    var total = 0L;
    for (var risk : RISKS) {
        total += risk.base().cents();
    }
    return total;
}
```

Money inline in Risk: true, Risk on the heap: 24 B, buffered Money would be: 24 B

=== the same loop over 10 000 risks, batch after batch ===

batch 1	(calls 1- 20)	24,01 B per read	4 801 256 B total	
batch 2	(calls 21- 40)	24,01 B per read	4 801 208 B total	
batch 3	(calls 41- 60)	6,00 B per read	1 200 848 B total	
batch 4	(calls 61- 80)	0,00 B per read	728 B total	<- C2 has compiled it: no Money is built any more
batch 5	(calls 81-100)	0,00 B per read	728 B total	<- C2 has compiled it: no Money is built any more
batch 6	(calls 101-120)	0,00 B per read	728 B total	<- C2 has compiled it: no Money is built any more
batch 7	(calls 121-140)	0,00 B per read	728 B total	<- C2 has compiled it: no Money is built any more
batch 8	(calls 141-160)	0,00 B per read	728 B total	<- C2 has compiled it: no Money is built any more
batch 9	(calls 161-180)	0,00 B per read	728 B total	<- C2 has compiled it: no Money is built any more
batch 10	(calls 181-200)	0,00 B per read	728 B total	<- C2 has compiled it: no Money is built any more
batch 11	(calls 201-220)	0,00 B per read	728 B total	<- C2 has compiled it: no Money is built any more
batch 12	(calls 221-240)	0,00 B per read	728 B total	<- C2 has compiled it: no Money is built any more

the whole warm-up cost once, per JVM: 10 803 312 B of young-gen garbage
 what it buys, for as long as the data lives: 160 000 B and 10 000 objects saved
 (an identity Money would be a separate 16 B object per Risk – and free to read, always)

W C1 ten odczyt kosztuje alokację
 Alokacja znika w C2

PODSUMOWANIE



„Premature optimization is the root of all evil”

- Donald Knuth

„Trzeba uważać co się robi”

- Leszek

- **EP 401: Value Classes and Objects:** <https://openjdk.org/jeps/401>
- **JEP 539: Strict Field Initialization in the JVM:** <https://openjdk.org/jeps/539>
- **Project Valhalla** (status JEP-ów i szkice, m.in. null-restricted): <https://openjdk.org/projects/valhalla/>
- **Buildy early access:** <https://jdk.java.net/valhalla/>
- **Source code tej prezentacji** <https://github.com/LeszekLeszczynski/confitura2026>

DZIĘKUJĘ ZA UWAGĘ

